

JOB TRACKER

Deployment & System Test Document

CSCE 548 — Database Systems

University of South Carolina

1. Project Overview

The Job Tracker is a full-stack n-tier web application for tracking job applications, companies, contacts, job postings, and application pipeline statuses. It is organized into four layers:

- Data Layer — Python DataProvider class using mysql-connector-python
- Business Layer — Python BusinessManager class
- Service Layer — Python FastAPI REST API served by Uvicorn
- Client Layer — React (JavaScript) single-page application served by Vite

All source code is in a single GitHub repository with two top-level folders: backend/ (Python) and frontend/ (React).

2. Prerequisites

Install all of the following before attempting to run the project. All tools are free.

Software	Version	Download	Notes
Python	3.11+	python.org/downloads	Check 'Add to PATH' during install
Node.js	18 LTS+	nodejs.org	Includes npm. Run: <code>node --version</code>
MySQL	8.0+	dev.mysql.com/downloads	Community Edition is free
MySQL Workbench	8.0+	dev.mysql.com/downloads	Recommended for DB verification
Git	Any	git-scm.com	For cloning the repo
VS Code	Any	code.visualstudio.com	Recommended IDE

Note: Windows: During Python installation, check 'Add Python to PATH'. During MySQL installation, note the root password you set — you will need it later.

3. Download the Project

Option A — Clone with Git (recommended)

PowerShell or CMD:

```
git clone https://github.com/kcrasborn/CSCE-548-Database.git
cd CSCE-548-Database
```

Option B — Download ZIP from GitHub

1. Go to the GitHub repository page
 2. Click the green Code button and select Download ZIP
 3. Extract the ZIP to a folder on your machine
 4. Open a terminal inside the extracted CSCE-548-Database folder
-

4. Database Setup

4.1 Create and seed the database

Run the following commands from inside the backend/ folder. You will be prompted for your MySQL root password.

Step 1 — Create the schema:

```
mysql -u root -p < schema.sql
```

Step 2 — Insert sample data:

```
mysql -u root -p job_tracker < seed.sql
```

Note: If MySQL is not in your PATH, use the full path: C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe

4.2 Verify in MySQL Workbench (optional)

1. Open MySQL Workbench and connect to localhost with your root credentials
 2. Run: `SELECT * FROM company;` to confirm seed data loaded correctly
-

5. Backend Setup

5.1 Navigate to the backend folder

```
cd CSCE-548-Database\backend
```

5.2 Install Python dependencies

```
pip install -r requirements.txt
```

This installs: fastapi, uvicorn, pydantic, mysql-connector-python, requests, pytest.

5.3 Configure database credentials

Default credentials are in `app/dependencies.py`. Override them with environment variables if your MySQL uses different settings.

PowerShell:

```
$env:DB_HOST = "localhost"
$env:DB_USER = "root"
$env:DB_PASSWORD = "your_password_here"
$env:DB_NAME = "job_tracker"
$env:DB_PORT = "3306"
```

Command Prompt:

```
set DB_HOST=localhost
set DB_USER=root
set DB_PASSWORD=your_password_here
set DB_NAME=job_tracker
set DB_PORT=3306
```

Note: Environment variables set in a terminal session last only for that session. Set them before running `uvicorn` in the same terminal window.

5.4 Start the API server

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

You should see:

```
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Application startup complete.
```

Note: Keep this terminal open. The `--reload` flag auto-restarts the server when you save a file.

6. Frontend Setup

6.1 Open a second terminal and navigate to frontend

```
cd CSCE-548-Database\frontend
```

6.2 Install Node dependencies

```
npm install
```

Installs React, Vite, axios, and react-router-dom into a local `node_modules/` folder. Only needs to run once.

6.3 Start the React development server

```
npm run dev
```

You should see:

```
VITE v5.x.x ready in NNN ms
```

```
Local: http://localhost:3000/
```

Note: The frontend proxies all /api calls to `http://localhost:8000` automatically via `vite.config.js`. Both `uvicorn` and `npm run dev` must be running simultaneously.

7. Verifying the Setup

After starting all three components, verify each one individually:

Component	How to verify	Expected result
MySQL	Open MySQL Workbench and connect	Connection succeeds, tables visible
Backend API	Open <code>http://localhost:8000/</code>	JSON: <code>{"status": "ok"}</code>
API Docs	Open <code>http://localhost:8000/docs</code>	Swagger UI loads with all endpoints
Frontend	Open <code>http://localhost:3000/</code>	Dashboard loads with kanban board
Full stack	Click any navigation tab	Data appears from database

7.2 Verify API endpoints directly

With only the backend running, open these URLs in a browser to get raw JSON:

- `http://localhost:8000/api/companies`
- `http://localhost:8000/api/contacts`
- `http://localhost:8000/api/job-postings`
- `http://localhost:8000/api/applications`
- `http://localhost:8000/api/application-statuses`

8. Layer Testing with demo.py

The `demo.py` script tests all three layers independently. Run it from inside the `backend/` folder.

8.1 Test the Data Layer (DataProvider)

PowerShell:

```
$env:LAYER = "data"
```

```
python demo.py
```

8.2 Test the Business Layer (BusinessManager)

PowerShell:

```
$env:LAYER = "business"
python demo.py
```

8.3 Test the Service Layer (requires uvicorn running)

PowerShell:

```
$env:LAYER = "service"
python demo.py
```

Note: Each test run creates [TEST] records in the database. ApplicationStatus records are deleted at the end. All other records are left for visual verification.

8.4 Run Pytest

Business layer:

```
$env:LAYER = "business"
pytest -v
```

Data layer:

```
$env:LAYER = "data"
pytest -v
```

9. API Endpoints Reference

All endpoints follow REST conventions. Base URL: <http://localhost:8000>. Full interactive docs at <http://localhost:8000/docs>.

Method	Endpoint	Description
GET	/api/companies	Get all companies
GET	/api/companies/{id}	Get company by ID
POST	/api/companies	Create a company
PUT	/api/companies/{id}	Update a company
DELETE	/api/companies/{id}	Delete a company
GET	/api/contacts	Get all contacts
GET	/api/contacts/{id}	Get contact by ID
POST	/api/contacts	Create a contact

PUT	/api/contacts/{id}	Update a contact
DELETE	/api/contacts/{id}	Delete a contact
GET	/api/job-postings	Get all job postings
GET	/api/job-postings/{id}	Get job posting by ID
POST	/api/job-postings	Create a job posting
PUT	/api/job-postings/{id}	Update a job posting
DELETE	/api/job-postings/{id}	Delete a job posting
GET	/api/applications	Get all applications
GET	/api/applications/{id}	Get application by ID
POST	/api/applications	Create an application
PUT	/api/applications/{id}	Update an application
DELETE	/api/applications/{id}	Delete an application
GET	/api/application-statuses	Get all statuses (nested objects expanded)
GET	/api/application-statuses/{id}	Get status by ID
POST	/api/application-statuses	Create a status
PUT	/api/application-statuses/{id}	Update a status
DELETE	/api/application-statuses/{id}	Delete a status

10. Common Errors and Fixes

Problem	Fix
'python' not recognized	Reinstall Python and check 'Add to PATH', or use 'py' instead of 'python'
'uvicorn' not recognized	Run: pip install uvicorn. Or: python -m uvicorn main:app --reload
'npm' not recognized	Node.js not installed. Reinstall from nodejs.org
Error loading ASGI app	Wrong folder. Run: cd backend, then retry uvicorn
MySQL Connection not available	MySQL not running. Open Windows Services and start MySQL80
Access denied for user root	Wrong password. Set \$env:DB_PASSWORD to your MySQL root password
Port 8000 already in use	Run: netstat -ano findstr :8000, kill the PID, or use --port 8001
Port 3000 already in use	Edit vite.config.js: server: { port: 3001 }

Frontend shows no data	Uvicorn is not running. Start backend first, then refresh browser
LAYER=x python demo.py fails	Use PowerShell: \$env:LAYER = "service" then python demo.py
mysql command not found	Add MySQL bin to PATH or use full path to mysql.exe
Pylance import warnings	Editor-only warnings, not runtime errors. Add pyrightconfig.json to repo root

11. Project Structure

```

CSCE-548-Database/
README.md <- GitHub readme
.gitignore
pyrightconfig.json <- VS Code Pylance config
backend/ <- Python FastAPI application
main.py <- FastAPI entry point
demo.py <- Layer testing script
requirements.txt
schema.sql <- Database schema
seed.sql <- Sample data
app/
models.py <- Domain objects
data_provider.py <- Data layer
business_manager.py <- Business layer
schemas.py <- Pydantic API models
dependencies.py <- FastAPI DI (per-request connection)
routers/ <- Service layer (one file per controller)
companies.py / contacts.py / job_postings.py
applications.py / application_statuses.py
tests/
test_core.py <- Pytest CRUD tests
frontend/ <- React application
vite.config.js <- Dev server + /api proxy to port 8000
src/
api/client.js <- All API calls (axios)
components/ <- Navbar, Modal, Toast, StatusBadge
pages/ <- Dashboard, Pipeline, Applications, etc.

```

12. Quick Start Reference

After the one-time setup above, use these commands every session:

Terminal 1 — Backend

```
cd CSCE-548-Database\backend
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Terminal 2 — Frontend

```
cd CSCE-548-Database\frontend
npm run dev
```

Open in browser

```
http://localhost:3000 <- Full application UI
http://localhost:8000/docs <- Swagger API documentation
```

Note: *MySQL must be running as a Windows background service before starting either terminal.*